ChatGPT

# MERN-Stack Final-Year Project Opportunities for a BIT Student in Nepal

## Executive Summary

For a BIT final-year project in Nepal, the strongest MERN application is not another generic e-commerce site, learning-management system, hospital appointment page, or complaint portal. Nepal already has government digital systems that cover parts of these functions: the national BIPAD disaster platform exposes an API; the Ministry of Health operates a National Health Facility Registry with search, maps, dashboard functions and API-related access; CEHRD operates IEMIS for school information; Hello Sarkar accepts and tracks grievances; the National Statistics Office publishes local-level census profiles; Nepal Rastra Bank publishes daily foreign-exchange and payment-system statistics; and Kathmandu Metropolitan City already links services such as grievances, online tax and a geoportal. A higher-value final-year project should therefore **connect, enrich, localize or operationalize existing data rather than simply reproduce an existing portal**. [1]

This research evaluates fourteen project concepts across disaster management, transport, agriculture, health, education, governance, local commerce and finance. The assessment uses three criteria: **feasibility for a single BIT student, 40%; expected real-world impact, 35%; and novelty, 25%**. The scores are analytical estimates rather than empirical measurements.

The strongest three are:

| Rank | Project | Central proposition | Why it stands out |
|---|---|---|---|
| **1** | **SafeRoute Nepal** | Disaster-aware road-condition and route-risk web app | Combines authoritative BIPAD hazards/incidents/ highway/environmental feeds with geotagged community road reports. It is highly demonstrable and does not require a hospital, bank or municipality to give the student privileged data. |
| **2** | **AgroLink Nepal** | Cooperative produce aggregation, market intelligence and transport pooling | Tackles several connected problems—small lots, buyer discovery, transport coordination, reference prices and produce-safety information—in one workflow rather than making another farmer marketplace. |
| **3** | **ClinicBridge Nepal** | Health-facility service-availability and referral coordination | Builds on the existing national facility registry rather than competing with it, adding the missing operational layer of “where can this patient actually obtain this service now?” |

**SafeRoute Nepal is the recommended final-year project.** Its technical scope is unusually well matched to MERN: MongoDB supports GeoJSON and `2dsphere` geospatial indexing for proximity and intersection queries, while the BIPAD platform provides an official Nepal-specific API surface containing disaster-oriented datasets. That creates a credible project involving external API integration, geospatial databases, crowdsourcing, moderation, offline/PWA behavior, role-based access, mapping and analytics without needing machine learning simply for novelty. [2]

**AgroLink Nepal is arguably the best alternative for an agriculture-oriented student.** The Kalimati Fruit and Vegetable Market Development Board maintains an official government-market website and publishes downloadable pesticide-residue analysis notices, including extensive 2026 publications. However, this research did not verify a stable public JSON price API from Kalimati, so a robust student project should permit admin CSV/manual market-price ingestion and treat automated extraction as an optional integration requiring permission and a stable source. [3]

**ClinicBridge Nepal may have the highest social impact but carries more execution risk.** Nepal's National Health Facility Registry already supplies the authoritative facility-directory layer; the proposed application adds frequently updated service availability, referral requests and facility-side workflow. For a public academic deployment, referrals should use synthetic data and avoid diagnoses or detailed medical records unless a real health institution formally supports the pilot. [4]

A zero-dollar or nearly zero-dollar academic prototype is technically possible before optional domain, messaging, map-provider or file-storage costs: Vercel currently lists its Hobby tier at **$0/month**, Render provides free Node-capable web services for testing/hobby use, and MongoDB Atlas offers a free cluster with **0.5 GB storage and a 100-read/write-operations-per-second limit**. The trade-off is important: Render explicitly says its free services should not be used for production and spins an idle free service down after 15 minutes, taking roughly a minute to restart. [5]

**Assumptions.** The recommendations assume one BIT student, MERN as the core stack, an unspecified budget, a flexible overall academic timeline, and roughly three months for the selected implementation. External institutional approvals are not assumed. “Novelty” means differentiation from the public systems identified in this research; it is **not** a claim that no private developer or startup in Nepal has ever attempted something similar.

## Nepal Context, Assumptions, and Evaluation Method

Several characteristics of Nepal's current digital-data environment make integration-oriented projects particularly attractive.

First, Nepal has meaningful public digital infrastructure that a final-year project can build around. BIPAD has an official API rather than only a human-readable portal. The National Health Facility Registry provides national facility discovery, maps and dashboard functions. CEHRD has an Integrated Educational Management Information System and a separate government learning platform. NSO's census system allows navigation down to province, district and municipality. NRB exposes official daily exchange rates, key payment-system indicators and financial-inclusion resources. [6]

Second, **not every useful official source is a developer-ready API**. Kalimati's current pesticide-residue publications, for example, are visible as dated downloadable notices, while no stable public JSON market-price API was verified in this research. The architectural implication is important: every serious project should isolate external data behind an **adapter layer** and maintain local cached/normalized data. The final viva should still work with a seeded fixture dataset if an upstream system becomes unavailable. [7]

Third, duplication should be treated as a novelty penalty. Hello Sarkar already lets citizens lodge and check the status of complaints through the Office of the Prime Minister and Council of Ministers, using online and several other channels. Kathmandu Metropolitan City's website similarly links grievance and geoportal services. Therefore, “citizen complaint system” alone would be a weak final-year idea; **asset maintenance, SLA measurement, ward work orders, geospatial evidence and public performance dashboards** create much clearer differentiation. [8]

The scoring rubric used below is:

| Dimension | Weight | What raises the score | What lowers it |
|---|---|---|---|
| **Feasibility** | **40%** | Public/obtainable data, bounded MVP, straightforward validation, low institutional dependency | Restricted data, hardware dependence, specialized algorithms, regulatory/privacy burden |
| **Impact** | **35%** | Clear stakeholder pain, measurable operational benefit, relevance outside a classroom demo | Convenience-only functionality or weak path to real adoption |
| **Novelty** | **25%** | Nepal-specific workflow, useful integration of fragmented data, distinctive multi-user process | Generic CRUD marketplace/LMS/ booking/complaint clone |

The weighted score is:

$$\text{Score} = 0.40F + 0.35I + 0.25N$$

The scoring intentionally favors **finishability over technical spectacle**. A final-year project receives more value from a complete system with users, tests, data provenance and measurable outcomes than from an unfinished AI feature.

A second design principle is **offline tolerance**. It does not require rebuilding the entire application offline: cache critical read views, save draft forms locally and queue user-generated reports until the network returns. This particularly benefits disaster, agriculture, field-audit and transport concepts.

A third principle is **minimum necessary data**. Projects involving health, education or finance should demonstrate authorization, auditability and privacy without creating unnecessary collections of diagnoses, citizenship documents, children's personal information or bank credentials. The recommended health MVP, for example, is a referral coordination system rather than an electronic health record.

## Project Portfolio

**SafeRoute Nepal — disaster-aware road access and route-risk intelligence.**
**Problem statement:** During floods, landslides and other disruptions, a user may need both authoritative disaster information and very local knowledge about whether a specific road section is obstructed. BIPAD already exposes disaster-oriented API resources; the opportunity is to turn these into a focused mobility workflow rather than reproduce the whole disaster portal. [9] **Target users:** residents, travelers, delivery drivers, local responders, ward personnel and NGOs. **Core features:** interactive hazard/incident map; road-condition reports with photo/location/time; verification voting; report expiry; BIPAD overlays; route/corridor risk assessment; subscriptions for selected areas; moderation dashboard; offline draft reporting. **Data flow:** BIPAD adapter → normalization job → MongoDB GeoJSON collections → Express geospatial queries → React map; citizen → report → moderation/verification → confidence score → map/alert service. MongoDB's geospatial operators support proximity, containment and geometry-intersection queries through geospatial indexes. [10] **Integrations/APIs:** BIPAD alerts/incidents/hazards/rain/river/weather/highway resources; OSM-compatible mapping; carefully throttled Nominatim for geocoding. The public Nominatim service imposes a maximum of one request per second, requires attribution/identification and forbids client-side autocomplete, so geocoding must be cached and abstracted behind a provider interface. [11] **Complexity:** Medium–High. **Novelty/impact:** Very high—the strongest combination of Nepal-specific data, technical depth and plausible public benefit in this portfolio. **Milestones:** architecture, BIPAD proof-of-concept and map prototype, **1.5 weeks**; reporting, verification and geospatial risk engine, **3.5 weeks**; offline support, testing, admin dashboard and deployment, **3 weeks**.

**AgroLink Nepal — cooperative produce aggregation, buyer matching and transport pooling.**
**Problem statement:** A simple farmer marketplace leaves difficult coordination work unresolved: produce is available in separate small lots, buyers want predictable quantities/grades, transport should be shared, and farmers need market/safety reference information. **Target users:** small farmers, cooperatives, vegetable collectors, institutional buyers, retailers and transporters. **Core features:** harvest forecasts and produce lots; cooperative lot aggregation; buyer demand requests; matching by product/quantity/location/date; reference market-price history; pooled transport capacity; order status; quality/pesticide notice feed; optional payment deposit. Kalimati's official market system publishes current downloadable pesticide-residue analysis results, providing a credible local quality-data source. [12] **Data flow:** farmer lots → cooperative aggregation → buyer request → match service → order → pooled trip/booking; official market publications → admin/import adapter → normalized market/quality records → dashboard. **Integrations/APIs:** Kalimati publications or approved datasets; BIPAD weather-related resources where useful; maps/geocoding; optional Khalti payment initiation. Khalti documents its gateway as an API-based online-payment solution specifically for Nepal. [13] **Complexity:** Medium–High. **Novelty/impact:** High to very high; stronger than “farmers sell vegetables online” because aggregation and logistics are the central innovation. **Milestones:** interviews, commodity model, data-import prototype, **1.5 weeks**; lot/demand/matching/order workflows, **4 weeks**; transport pooling, quality feed, analytics and deployment, **3 weeks**.

**ClinicBridge Nepal — facility availability and referral coordination.**
**Problem statement:** A directory can tell a user that a facility exists, but an operational referral requires knowing which service is available, how recently that status was updated and whether the destination will accept the referral. Nepal already has an official National Health Facility Registry, making it preferable to use authoritative facility identity/location data rather than create another unverified hospital directory. [4] **Target users:** patients, health posts, primary-care staff, hospitals and referral coordinators. **Core features:** service-based facility search; facility profile; service/queue availability with freshness timestamp; referral

request; accept/redirect/complete workflow; appointment request; staff dashboard; audit log. **Data flow:** NHFR/public facility seed → local facility collection → facility staff availability updates → public search; origin/referrer → minimum-data referral → destination → status event → tracking view. **Integrations/APIs:** NHFR where access is approved; maps/geocoding; optional SMS/email later. **Complexity:** High because institutional onboarding and health-data handling are harder than the code alone. **Novelty/impact:** Very high, provided the project remains an operational coordination tool rather than another appointment site. **Milestones:** scope, service taxonomy and NHFR proof-of-concept, **2 weeks**; directory, staff availability and referral state machine, **4 weeks**; audit/security, testing, synthetic-data pilot and deployment, **3 weeks**.

**Accessible Nepal — crowdsourced accessibility map for essential services.**
**Problem statement:** A facility's existence does not indicate whether a wheelchair user or someone with limited mobility can actually reach the entrance, toilet, service counter or relevant floor. **Target users:** persons with disabilities, caregivers, health-service users, accessibility advocates, local governments and facility managers. **Core features:** structured accessibility audit; ramp/step/elevator/toilet/doorway/service-counter fields; photographs; "last verified" status; facility response; accessible-place search and map; moderation; issue history rather than a simplistic single "accessible/not accessible" score. **Data flow:** official facility/school seed → community audit → evidence/moderation → normalized accessibility attributes → map/filter → facility correction/update. **Integrations/APIs:** NHFR for health-facility identities; CEHRD/IEMIS-derived institution references where access permits; NSO local-level demographic context; OSM geospatial data. [14] **Complexity:** Medium. **Novelty/impact:** High, particularly because the data produced by the application itself is valuable. **Milestones:** checklist/schema/map, **1 week**; auditing, photo evidence and moderation, **3 weeks**; verification, analytics, PWA and deployment, **2.5 weeks**.

**WardWorks Nepal — municipal asset maintenance and SLA transparency.**
**Problem statement:** Building another grievance form adds little because Hello Sarkar already supports grievance registration and status tracking, and municipalities such as Kathmandu expose their own digital-service links. [8] The more defensible problem is what happens **after** a pothole, blocked drain, broken streetlight or damaged public asset is reported. **Target users:** residents, ward offices, municipal engineers, maintenance teams and contractors. **Core features:** geotagged asset registry; issue linked to an asset; triage/severity; assignee/work order; expected service time; before/after evidence; reopening; maintenance history; ward-level SLA dashboard; public anonymized metrics. **Data flow:** report → duplicate/nearby-asset check → ward triage → work order → completion evidence → citizen confirmation → KPI aggregation. **Integrations/APIs:** maps; municipal ward directory; optional link/reference field for an existing Hello Sarkar ticket rather than replacing Hello Sarkar. MoFAGA's official site provides local-government navigation/contact resources. [15] **Complexity:** Medium. **Novelty/impact:** High if asset lifecycle and accountability—not complaints—are emphasized. **Milestones:** asset/issues model and map, **1 week**; work-order/SLA workflow, **3.5 weeks**; dashboards, evidence, tests and pilot, **2.5 weeks**.

**Farm Sentinel Nepal — community crop-pest and disease observation network.**
**Problem statement:** An individual farmer seeing a crop symptom is more useful to an extension service when that observation becomes a geotagged, time-stamped pattern that can be reviewed by a knowledgeable person. **Target users:** farmers, cooperatives, agriculture technicians and municipal agriculture sections. **Core features:** crop/issue reports with images; location privacy at adjustable precision; nearby report clustering; technician review; verified/unverified state; crop-specific alerts; discussion/follow-up; seasonal heat map. **Data flow:** farmer observation → media/storage + MongoDB location → spatial/time cluster → technician queue → verification → localized notification. **Integrations/APIs:** BIPAD weather/environmental inputs where relevant; optional ICIMOD land-cover datasets for research/visualization.

ICIMOD's regional database includes annual Nepal land-cover data produced through its land-cover monitoring work. [16] **Complexity:** Medium–High. **Novelty/impact:** High. The MVP should **not** claim to diagnose plant disease from an image; a verified observation network is both safer and achievable. **Milestones:** reporting taxonomy/geospatial model, **1.5 weeks**; clustering/reviewer workflow, **4 weeks**; alerts, map analytics, PWA and field test, **3 weeks**.

**Relief Inventory Exchange — disaster-stock and needs matching.**
**Problem statement:** During an emergency, inventory and needs can be fragmented across wards, NGOs and warehouses; the useful software problem is not merely "donation registration" but matching the right item, quantity and location while preserving an auditable dispatch trail. **Target users:** municipalities, wards, NGOs, community organizations and relief warehouses. **Core features:** incident-linked needs; inventory lots; expiry dates; reservation; fulfillment matching; QR/dispatch acknowledgement; duplicate-request flagging; warehouse map; shortage dashboard; audit trail. **Data flow:** BIPAD incident → local response workspace → verified need → inventory match → reservation → dispatch → receipt → remaining need/ inventory recalculation. BIPAD provides the authoritative integration anchor for disaster events. [9] **Integrations/APIs:** BIPAD, maps and optional QR scanning. **Complexity:** Medium. **Novelty/impact:** High impact, but somewhat less novel than SafeRoute because the primary challenge is workflow rather than novel geospatial synthesis. **Milestones:** incident/inventory/need model, **1.5 weeks**; matching and dispatch workflow, **3.5 weeks**; QR, analytics, audit tests and deployment, **2.5 weeks**.

**SchoolPulse Nepal — attendance-to-intervention case management.**
**Problem statement:** Recording attendance is less useful if recurring absences never produce a structured follow-up action. CEHRD already operates IEMIS, so a project should avoid trying to replace national EMIS functionality. [17] **Target users:** community-school teachers, head teachers, counselors/assigned focal staff and guardians. **Core features:** class attendance; configurable rule-based absence flag; intervention case; reason categories such as transport, seasonal work or health; guardian-contact log; action plan; follow-up dates; anonymous school dashboard; CSV import/export for authorized institutional workflows. **Data flow:** attendance → deterministic threshold/rule → teacher review → intervention case → actions → follow-up → aggregated trend. **Integrations/APIs:** IEMIS only with authorization; otherwise school-owned CSV; optional email/SMS. **Complexity:** Medium. **Novelty/impact:** Medium–High. Avoid opaque "AI dropout prediction"; the academically stronger MVP is an explainable workflow where staff make decisions. **Milestones:** attendance/import and permissions, **1.5 weeks**; case-management workflow/dashboard, **3.5 weeks**; notifications, privacy testing and pilot, **2.5 weeks**.

**Scholarship Navigator Nepal — eligibility and application-readiness engine.**
**Problem statement:** Scholarship information distributed as notices is difficult to turn into a personalized "am I eligible and what documents do I need?" workflow. **Target users:** SEE/+2/bachelor-level students, guardians, colleges and counselors. **Core features:** curated official opportunity catalog; rule-based eligibility; deadline calendar; document checklist; saved opportunities; verification/version history; multilingual summaries; notification preferences. CEHRD operates official digital education portals that can serve as source/verification points for education-related notices, while municipalities and institutions can be added through curated admin workflows. [18] **Data flow:** admin captures official notice → structured eligibility rules → reviewer verifies → user profile → rules engine → eligible/possibly eligible/not eligible explanation → checklist. **Integrations/APIs:** official notice feeds/pages where permitted, calendar/email. **Complexity:** Low–Medium. **Novelty/impact:** Medium–High; technically simpler than the top ideas, but exceptionally finishable. **Milestones:** opportunity/rule model and admin panel, **1.5 weeks**; eligibility/ checklist engine, **3 weeks**; alerts, accessibility, testing and launch, **2 weeks**.

**MeroBazaar Ledger — inventory, customer credit and digital receipts for micro-retail.**
**Problem statement:** A useful local-commerce application can focus on day-to-day microbusiness operations instead of competing with nationwide e-commerce marketplaces. **Target users:** kirana shops, small wholesalers and regular customers. **Core features:** inventory; purchase/sale entry; low-stock alerts; customer credit ledger; payment recording; digital receipt; supplier balances; daily cash-versus-credit summary; role-separated owner/staff accounts. **Data flow:** sale → inventory decrement + ledger event → receipt → daily aggregate; payment → outstanding balance update → immutable transaction history. **Integrations/APIs:** optional Khalti payment request/confirmation as a later milestone; Khalti's official gateway is designed for online web/mobile payments in Nepal. [13] **Complexity:** Medium. **Novelty/impact:** Medium—less novel than geospatial projects but very easy to test with real local businesses. **Milestones:** inventory/ledger model, **1 week**; sales/credit/receipt workflow, **3.5 weeks**; analytics, optional payment, backups and deployment, **2.5 weeks**.

**Cooperative Transparency — member ledger, decisions and audit trail.**
**Problem statement:** A savings/credit or multipurpose cooperative involves more than balances: members may need understandable statements and traceability from meetings/decisions to actions. For a student prototype, this should be an **information and governance layer**, not an unlicensed payment or lending platform. **Target users:** cooperative members, managers, accountants and board/audit personnel. **Core features:** member account statement; dues; loan/savings records imported from an approved source; meeting agendas/minutes; voting/approval records; document version history; audit log; discrepancy/issue tickets. **Data flow:** approved accounting import/manual entry → validation → member-scoped records → statement/dashboard; meeting → resolution → responsible officer → action status → audit trail. **Integrations/APIs:** CSV/accounting import; optional NRB public indicators only for contextual dashboards. NRB publishes payment-system, BFI and financial-inclusion data, although these are contextual rather than cooperative transaction feeds. [19] **Complexity:** High because correctness and privacy matter. **Novelty/impact:** High potential, but pilot access is the main risk. **Milestones:** domain/schema/audit design, **1.5 weeks**; member/ledger/governance workflows, **3.5 weeks**; reconciliation, permission tests and pilot, **3 weeks**.

**Public Transport Crowd ETA — passenger-sourced route reliability.**
**Problem statement:** Rather than attempting a full ride-hailing product, capture route and travel-time observations from ordinary passengers and turn them into route reliability information. **Target users:** bus/microbus commuters, operators and urban planners. **Core features:** route/stop map; “I am on this route” contribution; privacy-preserving GPS samples; next-vehicle approximation where enough samples exist; typical travel-time bands by time of day; crowding report; route correction/moderation; service reliability dashboard. **Data flow:** route baseline → opt-in anonymous observations → map matching/aggregation → ETA/reliability model → commuter display. **Integrations/APIs:** OSM mapping/geocoding; optional operator/municipal route data if obtained. Public Nominatim cannot be treated as an unlimited search service and specifically prohibits autocomplete, so the app should cache geocodes or use another provider as usage grows. [11] **Complexity:** High because GPS noise and the cold-start/user-adoption problem are substantial. **Novelty/impact:** High. **Milestones:** route model/map/data collection, **2 weeks**; aggregation/ETA prototype, **4 weeks**; privacy controls, PWA, evaluation and deployment, **3 weeks**.

**SkillBridge Local — verified student micro-project marketplace for local organizations.**
**Problem statement:** A normal job board does not solve the student's “I need real experience” problem or the small organization's “I need one bounded digital task, not a full-time hire” problem. **Target users:** BIT/CS/TVET students, local SMEs, schools, cooperatives, NGOs and municipalities. **Core features:** organizations

publish two-to-six-week scoped challenges; skill tags; applications; milestone acceptance; artifact submission; mentor feedback; organization-verified portfolio record; dispute/moderation workflow; institutional cohort dashboard. **Data flow:** organization → verified project → student application → milestone agreement → submission → approval/revision → verified portfolio evidence. **Integrations/APIs:** optional CEHRD/institution references; Git repository URL metadata without requiring GitHub API access; email. CEHRD's digital education ecosystem provides an official educational context but should not be treated as an identity API unless authorization is obtained. [20] **Complexity:** Low–Medium. **Novelty/impact:** Medium–High. **Milestones:** profiles/projects/applications, **1 week**; milestone/review/portfolio flow, **3 weeks**; organization verification, moderation and deployment, **2.5 weeks**.

**RemitWise Nepal — family remittance planning and FX-aware budgeting.**
**Problem statement:** Instead of handling remittances—which would substantially expand the regulatory/security scope—a student app can solve the information problem after or before a transfer: current official reference exchange rate, expected household budget allocation and savings-goal planning. **Target users:** migrant-worker households, students abroad and family financial planners. **Core features:** NRB exchange-rate reference; remittance scenario calculator; recurring family budget; goal envelopes; exchange-rate history visualization; “planned versus actual” household allocation; family-shared read-only plan; alerts. NRB's official website exposes daily exchange-rate resources alongside payment-system and financial-inclusion statistics. [19] **Data flow:** NRB reference-rate import → normalized rate history → user scenario → budget allocation → goal progress. **Integrations/APIs:** NRB official data through a permitted adapter/manual import; notification service. **Complexity:** Low–Medium. **Novelty/impact:** Moderate; very feasible but less distinctive than SafeRoute or AgroLink. **Milestones:** rates/scenario model, **1 week**; budgets/goals/history, **3 weeks**; sharing, alerts and testing, **2 weeks**.

## Prioritization and Ranked Recommendations

The following ranking uses the stated 40% feasibility, 35% impact and 25% novelty weights. Scores are my research-based project-selection judgment on a 1–5 scale, not measurements published by the cited organizations.

| Rank | Project | Sector | Feasibility | Novelty | Impact | Weighted / 5 | Main delivery risk |
|---|---|---|---|---|---|---|---|
| **1** | **SafeRoute Nepal** | Disaster + transport | 4.2 | 4.8 | 4.9 | **4.60** | Reliable community-report verification |
| **2** | **AgroLink Nepal** | Agriculture + commerce + transport | 4.3 | 4.5 | 4.7 | **4.49** | Stable market-data ingestion and multi-role scope |

| Rank | Project | Sector | Feasibility | Novelty | Impact | Weighted / 5 | Main delivery risk |
|---|---|---|---|---|---|---|---|
| **3** | **ClinicBridge Nepal** | Health | 3.9 | 4.6 | 4.9 | **4.43** | Institution/API access and sensitive-data handling |
| **4** | **Accessible Nepal** | Inclusion + governance + health | 4.5 | 4.2 | 4.3 | **4.36** | Building enough verified audit data |
| **5** | **WardWorks Nepal** | Governance | 4.5 | 3.9 | 4.4 | **4.32** | Getting a ward/ municipal pilot |
| **6** | **Farm Sentinel Nepal** | Agriculture | 3.9 | 4.5 | 4.5 | **4.26** | Expert verification and avoiding false diagnosis |
| **7** | **Relief Inventory Exchange** | Disaster management | 4.1 | 4.0 | 4.6 | **4.25** | Realistic relief-workflow validation |
| **8** | **SchoolPulse Nepal** | Education | 4.4 | 3.8 | 4.4 | **4.25** | Student privacy and school participation |
| **9** | **Scholarship Navigator** | Education + governance | 4.7 | 3.6 | 4.1 | **4.22** | Maintaining current verified opportunities |
| **10** | **MeroBazaar Ledger** | Local commerce + finance | 4.6 | 3.5 | 4.2 | **4.19** | Differentiation from existing POS/ledger apps |
| **11** | **Cooperative Transparency** | Finance + governance | 3.8 | 4.3 | 4.4 | **4.14** | Correctness, privacy and partner access |
| **12** | **Public Transport Crowd ETA** | Transport | 3.7 | 4.3 | 4.5 | **4.13** | Cold-start participation and noisy GPS |

| Rank | Project | Sector | Feasibility | Novelty | Impact | Weighted / 5 | Main delivery risk |
|---|---|---|---|---|---|---|---|
| **13** | **SkillBridge Local** | Education + commerce | 4.5 | 3.7 | 4.0 | **4.13** | Obtaining credible organizations/ projects |
| **14** | **RemitWise Nepal** | Finance | 4.6 | 3.4 | 3.8 | **4.02** | Moderate novelty |

A useful way to interpret the ranking is not that rank fourteen is a bad project. **RemitWise or Scholarship Navigator may actually be safer choices for a student who is still becoming comfortable with React and Node.** Conversely, SafeRoute's score reflects an unusually good balance between technical depth and achievable data access. BIPAD provides a real integration target, MongoDB directly supports the required geospatial operations, and a viable demonstration does not depend on convincing a hospital or local government to supply internal records. [2]

**Best project for academic marks and technical demonstration: SafeRoute.** It naturally demonstrates CRUD plus substantially more: background synchronization, external API handling, mapping, GeoJSON, indexed spatial queries, moderation, state/confidence logic, PWA caching and role-based administration.

**Best project for entrepreneurship potential: AgroLink.** It has several possible users who receive distinct value—farmer, cooperative, buyer and transporter—and can start as coordination software without holding payments. Kalimati's official publications provide credible Nepal-specific data to enrich the platform, and Khalti can be added only after the basic transaction workflow is solid. [21]

**Best project for social-service impact: ClinicBridge.** It benefits from an existing official facility registry, but the student should explicitly separate relatively stable **facility master data** from rapidly changing **availability data supplied by participating facilities**. [22]

**Projects to avoid in generic form:** an ordinary complaint portal, because Hello Sarkar already handles grievance registration and tracking; a plain hospital directory, because NHFR already exists; a generic school LMS, because CEHRD already operates a national learning platform; and a simple payment-enabled store where the technical contribution is only Khalti checkout. [23]

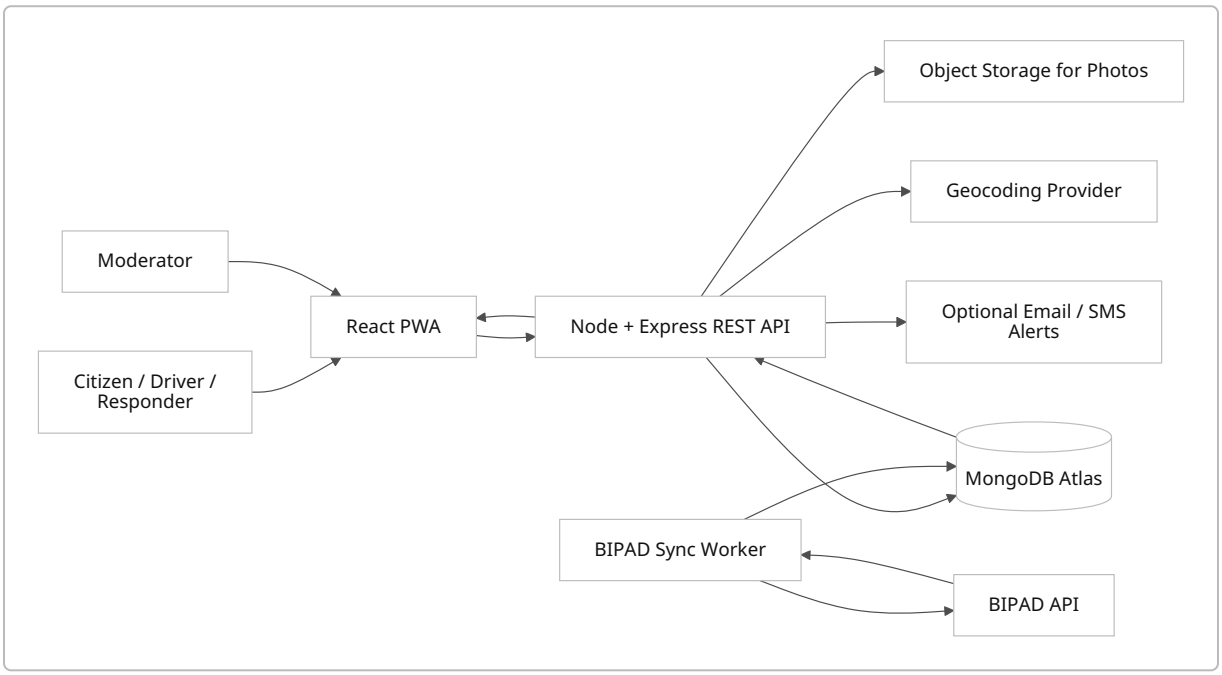
## Detailed Designs for the Top Three

### SafeRoute Nepal

**Recommended technical stack.** Use React + TypeScript with Vite, React Router, TanStack Query, React Hook Form and Zod on the client; MapLibre GL JS for map rendering and Turf.js for client-side geometric visualization; a service worker/IndexedDB queue for PWA behavior. Use Node.js, Express, TypeScript, Mongoose and Zod on the server. Add `node-cron` for small scheduled BIPAD imports; migrate to a queue only if the load justifies it. Testing should use Vitest/React Testing Library on the client, Supertest for the API and Playwright for key end-to-end flows.

MongoDB is particularly appropriate here because hazards and road reports can be stored as GeoJSON and indexed with `2dsphere`; MongoDB documents support `$geoIntersects`, `$geoWithin`, `$near` and `$geoNear`, which map directly to “hazards intersecting a corridor” and “reports near this location.” [10]

The authoritative-data adapter should periodically fetch relevant BIPAD records and normalize them. Keep the raw external ID and source timestamp so that an imported event can be updated idempotently rather than duplicated. [9]

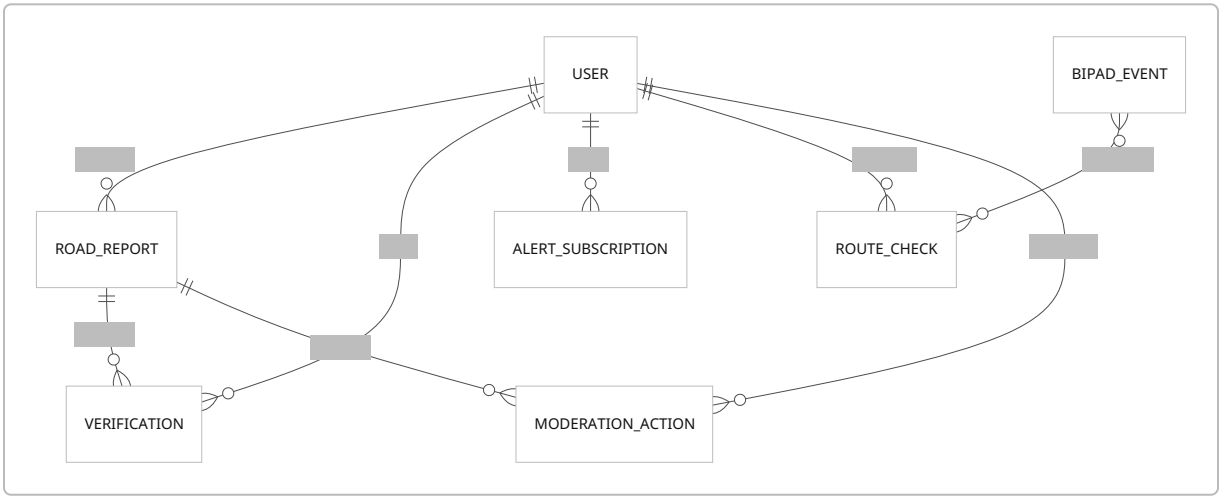



The public Nominatim service is suitable only for modest geocoding use under its policy: it caps use at one request per second, requires application identification and attribution, recommends caching, discourages recurring bulk geocoding and forbids client-side autocomplete. Therefore, SafeRoute should put geocoding behind a server-side provider adapter and cache normalized locations. [11]

**Collection sketch.**

| Collection | Important fields | Key indexes / relationships |
|---|---|---|
| `users` | `_id`, `name`, `email`, `passwordHash`, `role`, `trustLevel`, `status` | unique email |
| `bipadEvents` | `sourceId`, `eventType`, `severity`, `geometry`, `sourceUpdatedAt`, `startsAt`, `endsAt`, `rawHash` | unique `sourceId`; `geometry: 2dsphere`; time index |
| `roadReports` | `reporterId`, `category`, `severity`, `geometry`, `description`, `media[]`, `status`, `confidence`, `expiresAt` | `geometry: 2dsphere`; reporter/status/date |

| Collection | Important fields | Key indexes / relationships |
|---|---|---|
| `verifications` | `reportId`, `userId`, `vote`, `comment`, `createdAt` | unique `(reportId,userId)` |
| `routeChecks` | `userId?`, `origin`, `destination`, `corridor`, `hazardHits[]`, `riskBand`, `generatedAt` | optional TTL/history |
| `alertSubscriptions` | `userId`, `area`, `hazardTypes[]`, `minimumSeverity`, `channel` | `area: 2dsphere` |
| `moderationActions` | `actorId`, `reportId`, `action`, `reason`, `createdAt` | report/date |
| `syncRuns` | `source`, `startedAt`, `finishedAt`, `status`, `readCount`, `writeCount`, `error` | source/date |




This is a **logical ER view**. MongoDB implementation can embed small event arrays where appropriate instead of mechanically reproducing a relational schema.

**REST API design.** REST is preferable to GraphQL for this project because the client workflows are bounded and REST keeps authentication, caching and the viva explanation simpler.

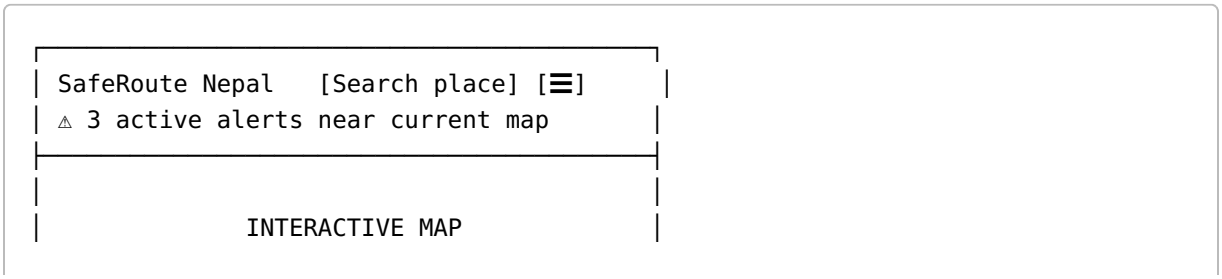
| Method | Endpoint | Purpose / authorization |
|---|---|---|
| `POST` | `/api/v1/auth/register` | Create normal user |
| `POST` | `/api/v1/auth/login` | Authenticate |
| `POST` | `/api/v1/auth/refresh` | Rotate/refresh session |

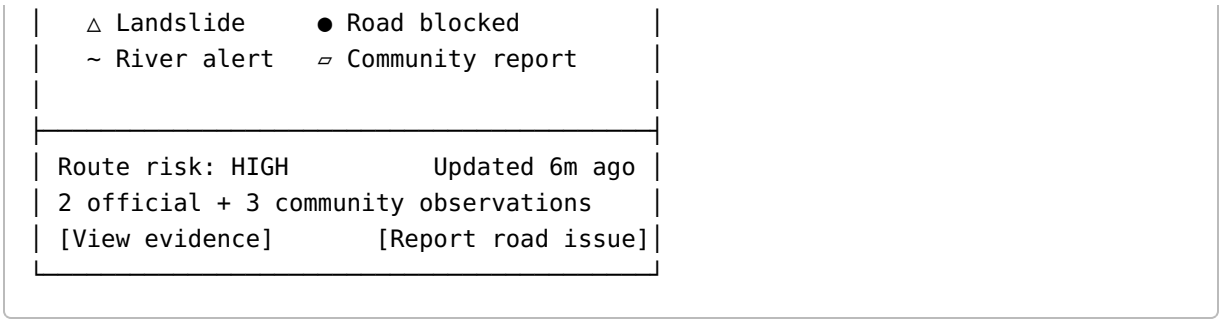
| Method | Endpoint | Purpose / authorization |
|---|---|---|
| `GET` | `/api/v1/hazards?bbox=&from=&severity=` | Public normalized BIPAD hazard layer |
| `GET` | `/api/v1/incidents?bbox=&from=` | Public incidents |
| `GET` | `/api/v1/road-reports?bbox=&status=` | Public verified/recent reports |
| `POST` | `/api/v1/road-reports` | Authenticated user submits report |
| `PATCH` | `/api/v1/road-reports/:id` | Owner before moderation or moderator |
| `POST` | `/api/v1/road-reports/:id/verifications` | Authenticated verification |
| `POST` | `/api/v1/routes/evaluate` | Evaluate supplied route/corridor against hazards/reports |
| `GET` | `/api/v1/routes/:id/risk` | Retrieve stored evaluation |
| `POST` | `/api/v1/alert-subscriptions` | Subscribe to an area/event type |
| `GET` | `/api/v1/admin/moderation-queue` | Moderator only |
| `PATCH` | `/api/v1/admin/reports/:id/status` | Moderator/admin |
| `GET` | `/api/v1/admin/sync-runs` | Admin/debug visibility |

The endpoint should return a **risk advisory**, not the statement “this route is safe.” A simple, explainable score can combine event severity, distance from the corridor, freshness and community confidence. A viva panel can inspect why a score occurred, which is preferable to introducing an opaque machine-learning model.

**Authentication and authorization.** Allow anonymous read-only map access. Use registered accounts for reporting and verification. Roles: `user`, `trustedReporter`, `moderator`, `admin`. Passwords are hashed; a short-lived access token plus a rotating refresh token in a Secure/HttpOnly cookie is a reasonable implementation. Add rate limiting to login and reporting, resource-ownership checks, email verification if practical, and an append-only moderation audit trail. Do not make SMS OTP mandatory in the student MVP because it creates recurring cost and another external dependency.

**UI wireframe description.**






On mobile, use a full-screen map with a draggable bottom sheet. The bottom sheet shows source, timestamp and evidence rather than crowding the map. An offline indicator should be persistent; unsent reports receive “Saved locally—pending upload” status.

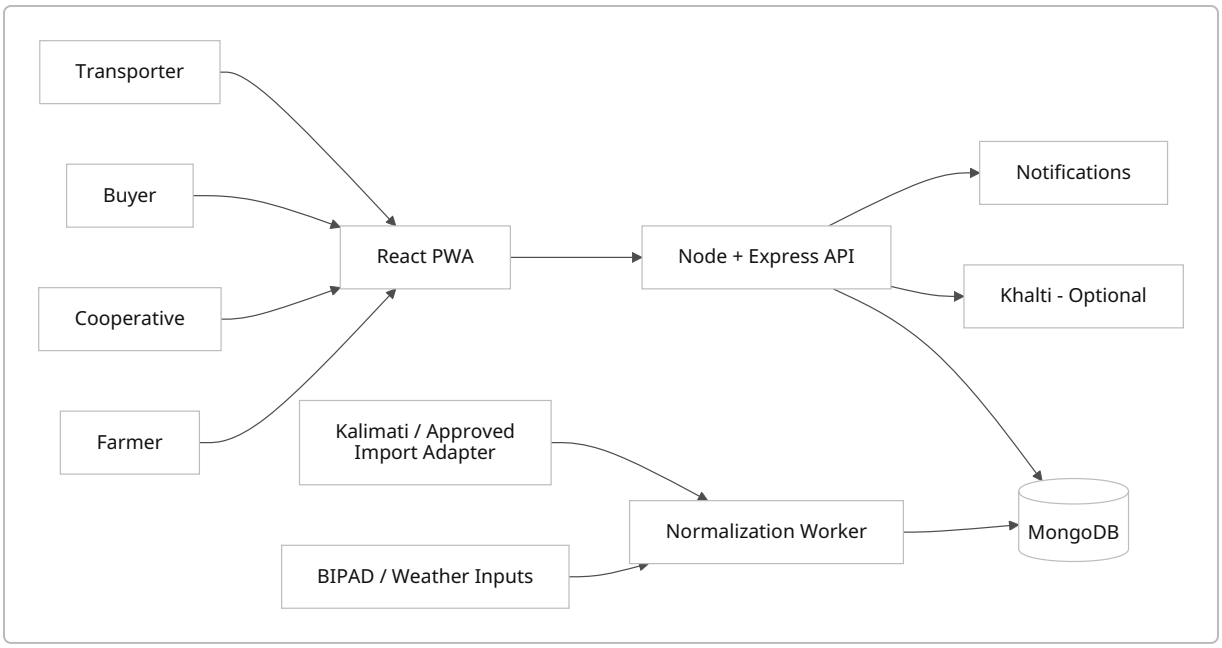
**Deployment.** For an academic prototype, host the React frontend on Vercel Hobby, the Express service on Render Free and MongoDB on Atlas Free. Current Vercel pricing lists Hobby at $0/month. Render supports free Node web services but explicitly classifies the tier as non-production and spins an idle service down after 15 minutes. Atlas's current free cluster permits 0.5 GB and 100 read/write operations per second, enough for a bounded academic dataset but not a national production service. [5]

For the viva, retain a local Docker/local-Node fallback plus a seeded database export so a hosting cold start or upstream outage cannot destroy the demonstration.

**AgroLink Nepal**

The architecture should deliberately avoid becoming five separate projects. Its core thesis is: **aggregate supply first, match demand second, consolidate transport third**. Payments and sophisticated forecasting are optional.

Use React/TypeScript + Vite, TanStack Query, React Hook Form/Zod, a PWA layer and a lightweight chart library. The backend can use Node/Express/TypeScript, Mongoose and scheduled import jobs. MongoDB GeoJSON is useful for farms, buyers and trips. Market-publication adapters should normalize commodity naming because “tomato,” varieties and Nepali/English spellings can otherwise fragment price histories.

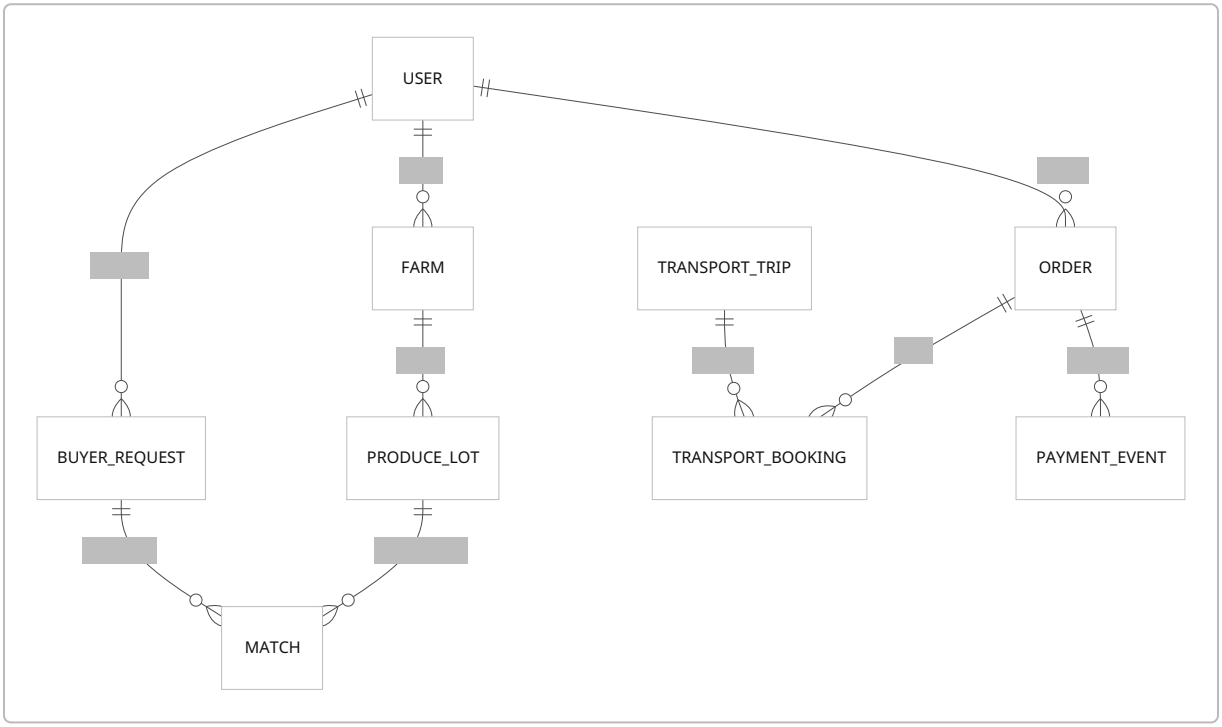



The Kalimati board's official website is a defensible local source, and its pesticide-residue section demonstrates that current market-quality reports are published as downloadable records. Because this research did not verify a stable public price API, **do not make web scraping a critical dependency**. Implement `MarketPrice` ingestion through manual CSV first, then replace the adapter if an authorized machine-readable source becomes available. [12]

**Collection sketch.**

| Collection | Important fields |
|---|---|
| `users` | role, name, contact, cooperativeId?, verificationStatus |
| `farms` | ownerId, cooperativeId?, name, GeoJSON location, crops[] |
| `produceLots` | farmId, commodity, variety, grade, quantity, unit, harvestDate, askPrice?, status |
| `marketPrices` | source, market, commodity, variety, date, min, max, average, ingestionMethod |
| `qualityNotices` | source, commodity, origin?, resultCategory, publishedAt, reference |
| `buyerRequests` | buyerId, commodity, quantity, minimumGrade, deliveryWindow, destination, maxPrice? |
| `matches` | requestId, lotIds[], scoreBreakdown, status |
| `orders` | buyerId, seller/cooperativeId, items[], value, deliveryStatus, paymentStatus |
| `transportTrips` | transporterId, origin, destination, capacityKg, departureAt, availableKg, status |

| Collection | Important fields |
|---|---|
| `transportBookings` | tripId, orderId, quantityKg, status |
| `paymentEvents` | orderId, provider, providerRef, state, receivedAt |




**REST endpoints.**

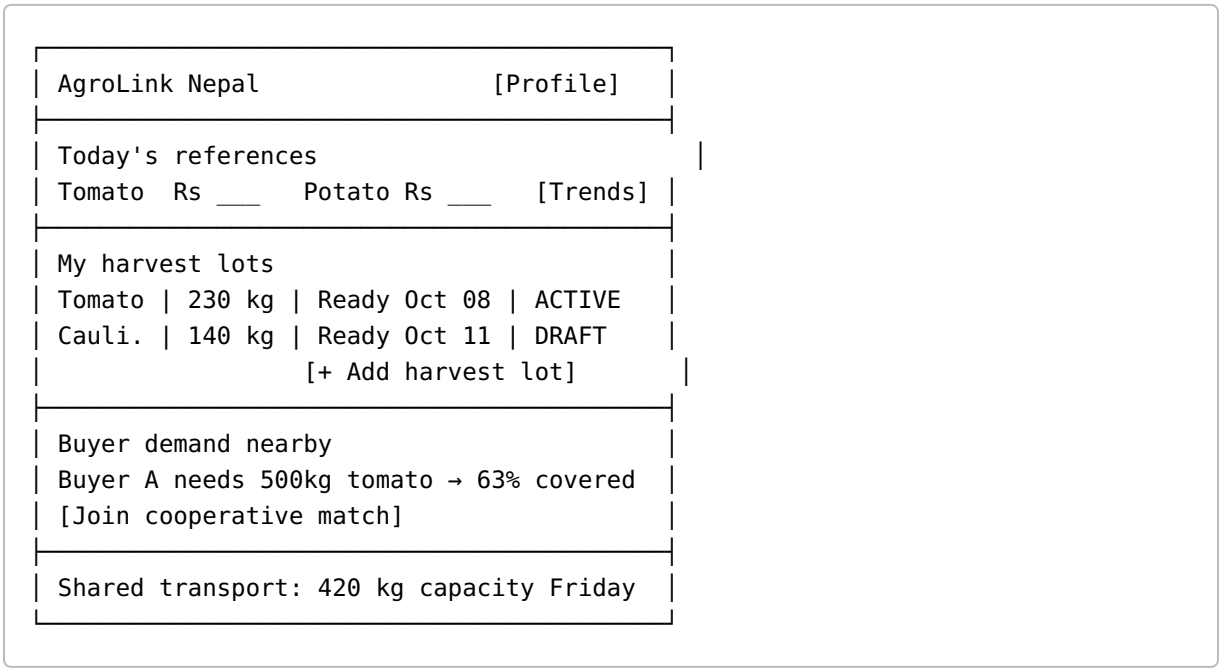
| Method | Endpoint | Purpose |
|---|---|---|
| `GET` | `/api/v1/market-prices?commodity=&from=&to=` | Reference-price history |
| `GET` | `/api/v1/quality-notices?commodity=` | Official/imported quality notices |
| `POST` | `/api/v1/lots` | Farmer creates lot |
| `GET` | `/api/v1/lots?commodity=&near=&date=` | Search supply |
| `PATCH` | `/api/v1/lots/:id` | Farmer/cooperative changes lot |
| `POST` | `/api/v1/buyer-requests` | Buyer posts requirement |
| `GET` | `/api/v1/buyer-requests/:id/matches` | Explainable candidate matches |
| `POST` | `/api/v1/orders` | Convert accepted match into order |
| `PATCH` | `/api/v1/orders/:id/status` | Controlled fulfillment transitions |

| Method | Endpoint | Purpose |
|---|---|---|
| `POST` | `/api/v1/transport-trips` | Transporter offers capacity |
| `GET` | `/api/v1/transport-trips?near=&to=&date=` | Discover trips |
| `POST` | `/api/v1/transport-trips/:id/bookings` | Reserve capacity |
| `POST` | `/api/v1/payments/khalti/initiate` | Optional deposit initiation |
| `POST` | `/api/v1/webhooks/khalti` | Optional provider callback |
| `POST` | `/api/v1/admin/import/market-prices` | Validated CSV/admin import |

Khalti's documentation explicitly provides a payment-gateway integration model for online services in Nepal, making it a credible optional stretch goal. It should come **after** lots, matching and order workflows; otherwise a student can spend too much of the semester debugging a feature that is not the core contribution. [13]

**Authentication/authorization.** Roles should be `farmer`, `cooperativeManager`, `buyer`, `transporter` and `admin`. Enforce both role and resource ownership: a farmer can edit only their own lots; a cooperative manager can aggregate lots belonging to their cooperative; a transporter cannot modify an order; a buyer cannot edit market-price imports. Verification status can be separate from role so unverified users can explore without being allowed to publish trusted listings.

**UI wireframe.**



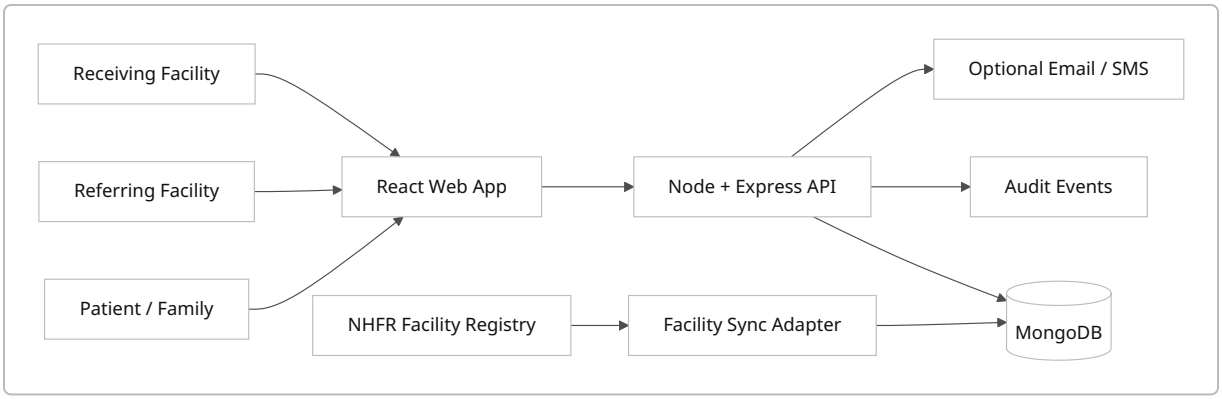

The buyer version reverses the emphasis: demand first, candidate aggregated lots second, transport third. A cooperative dashboard should show “requested 800 kg / confirmed 560 kg / shortage 240 kg,” because that aggregation logic is the project's main value proposition.

**Deployment.** The same Vercel/Render/Atlas academic stack is sufficient for a pilot. Avoid storing uploaded produce images directly on Render's filesystem because Render describes its free web-service filesystem as ephemeral; use object storage instead. [24]

**ClinicBridge Nepal**

ClinicBridge should be framed carefully: **NHFR remains the source of facility identity; ClinicBridge owns operational availability and referral workflow.** The official registry identifies itself as Nepal's National Health Facility Registry and exposes search/map/dashboard functionality; API-related access is part of the platform ecosystem. [4]

Use React/TypeScript + Vite, TanStack Query and accessible form components. Backend: Node/Express/TypeScript, Mongoose, Zod, a facility-import adapter and a deliberately explicit referral state machine. Do not add symptom diagnosis, prescription recommendations or AI triage to the final-year MVP.

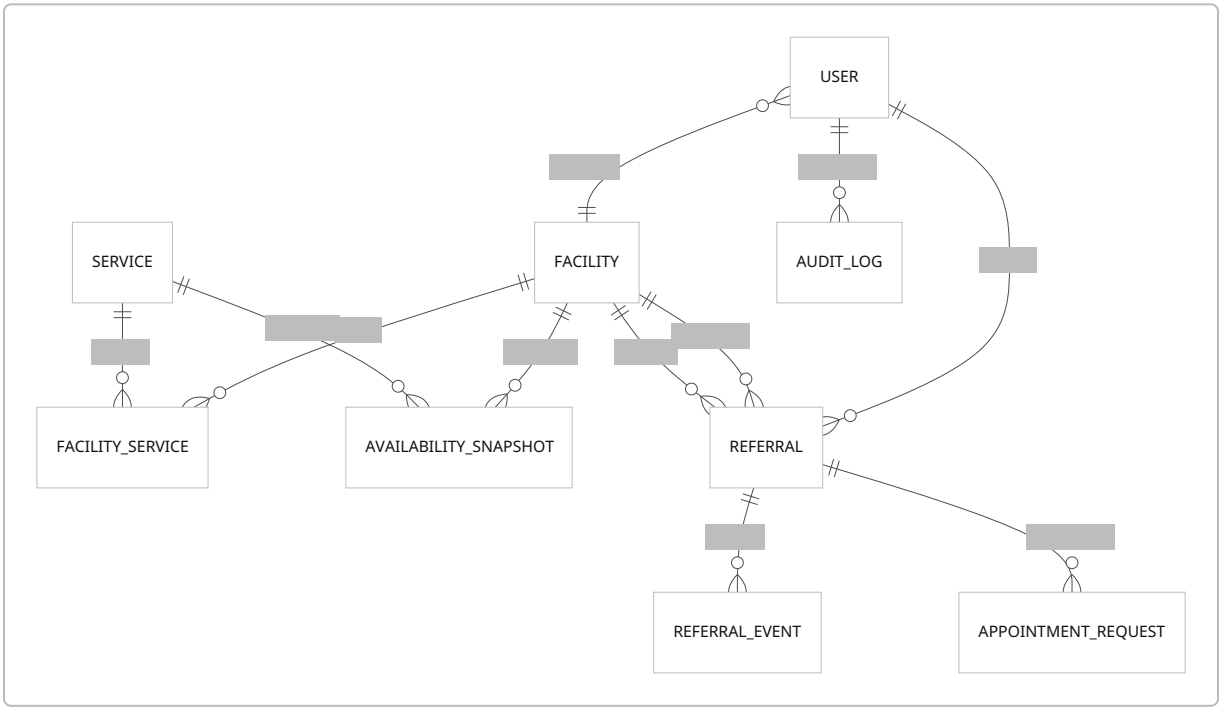



A key architectural rule is **freshness**. A facility's existence is stable master data; an “emergency service available” or “estimated queue band” observation is volatile. Every operational availability response should show who updated it and when, and stale data should automatically lose its “current” badge.

**Collection sketch.**

| Collection | Important fields |
|---|---|
| `users` | role, facilityId?, email, passwordHash, status, mfaEnabled |
| `facilities` | nhfrId, name, facilityType, address, GeoJSON location, contact, lastSyncedAt |
| `services` | code, name, category |
| `facilityServices` | facilityId, serviceId, enabled, schedule, lastConfirmedAt |
| `availabilitySnapshots` | facilityId, serviceId, availabilityBand, queueBand?, note?, updatedBy, expiresAt |
| `referrals` | trackingToken, originFacilityId?, destinationFacilityId, requestedServiceId, urgencyBand, status, minimalContact?, consentAt |

| Collection | Important fields |
|---|---|
| `referralEvents` | referralId, actorId, fromStatus, toStatus, reason, createdAt |
| `appointmentRequests` | referralId, requestedWindow, assignedSlot?, status |
| `auditLogs` | actorId, action, entityType, entityId, timestamp, metadata |




**REST endpoints.**

| Method | Endpoint | Purpose |
|---|---|---|
| `GET` | `/api/v1/facilities?near=&service=&type=` | Public facility search |
| `GET` | `/api/v1/facilities/:id` | Master facility profile |
| `GET` | `/api/v1/facilities/:id/services` | Offered services |
| `GET` | `/api/v1/facilities/:id/availability` | Current/stale operational snapshots |
| `PATCH` | `/api/v1/staff/facilities/:id/services` | Manager updates service offering |
| `POST` | `/api/v1/staff/availability` | Publish availability snapshot |
| `POST` | `/api/v1/referrals` | Create minimum-data referral |
| `GET` | `/api/v1/referrals/:trackingToken` | Authorized/token-based status view |

| Method | Endpoint | Purpose |
|---|---|---|
| `PATCH` | `/api/v1/referrals/:id/status` | Receiving/referring staff state transition |
| `POST` | `/api/v1/referrals/:id/appointments` | Request/assign appointment |
| `GET` | `/api/v1/staff/referrals/inbox` | Receiving facility queue |
| `GET` | `/api/v1/admin/audit` | Restricted audit trail |
| `POST` | `/api/v1/admin/facilities/sync` | Authorized registry synchronization |

**Authentication/authorization.** Public users should not need an account merely to search facilities. Staff accounts should be manually associated with an approved facility. Roles: `patient/user`, `facilityStaff`, `facilityManager`, `platformAdmin`. Facility staff see only referrals addressed to their facility; managers additionally manage service availability. Use MFA/TOTP for staff if time permits. The public academic instance should contain synthetic referrals and no real diagnoses, documents or detailed clinical notes.

**UI wireframe.**

```
┌──────────────────────────────────────────┐
│ ClinicBridge Nepal                       │
│ What service? [Emergency ▼]              │
│ Near [Bhaktapur________]       [Search]  │
├──────────────────────────────────────────┤
│ Facility A                2.1 km         │
│ Emergency: AVAILABLE  updated 12 min ago │
│ Lab:      LIMITED     updated 28 min ago │
│ [Details]                    [Refer]     │
├──────────────────────────────────────────┤
│ Facility B                4.8 km         │
│ Emergency: STALE       updated 9 hrs ago │
│ [Details]                                │
└──────────────────────────────────────────┘
```

The staff side should look more like an operational queue than a consumer booking site:

```
Incoming referrals: 7
[NEW 3] [ACCEPTED 2] [IN TRANSIT 1] [ARRIVED 1]

Service availability
Emergency        Available   Updated 12m ago
X-ray            Limited     Updated 22m ago
```

```
Ultrasound       Unavailable Updated 1h ago
                         [Update status]
```

**Deployment.** For a synthetic-data demonstration, the same free student stack is reasonable. It should not be presented as a production hosting recommendation for real patient-related data: Render itself states that its free instances are not for production, while Atlas free clusters impose storage, throughput and other resource limits. [25] A genuine health-facility pilot would require a separate security, governance, backup and institutional review beyond the final-year MVP.

## Datasets, Resources, and Three-Month Delivery Plan

**Recommended datasets and APIs**

| Source | Data/use | Relevant projects | Access strategy and caveats |
|---|---|---|---|
| **BIPAD Portal API — Government of Nepal** | Disaster alerts/events/ incidents/hazards plus environmental/ highway-oriented resources exposed through the API | SafeRoute, Relief Inventory, Farm Sentinel, AgroLink | **Primary choice for disaster data.** Consume through a backend adapter, cache locally and preserve source IDs/ timestamps. [9] |
| **National Health Facility Registry — MoHP** | Facility identities, search/map information and registry/API ecosystem | ClinicBridge, Accessible Nepal | Use as facility master data subject to available access; do not invent hospital identities. Availability remains locally supplied. [4] |
| **National Statistics Office Census Results** | Local-level demographic profiles by province/district/ municipality | Accessibility, governance, school/ agriculture dashboards | Good for aggregate contextual indicators, not for identifying individuals. The official census interface explicitly provides local-level selection. [26] |
| **CEHRD IEMIS** | Education-management information | SchoolPulse, education analytics | Integrate only through authorized access/export; no public developer API was verified in this research. [17] |
| **CEHRD Sikai Chautari** | Official learning resources and education ecosystem links | Scholarship Navigator, SkillBridge | Useful as a verified government source/reference; the platform identifies itself as a Government of Nepal education portal. [27] |

| Source | Data/use | Relevant projects | Access strategy and caveats |
|---|---|---|---|
| **Kalimati Fruit & Vegetable Market Development Board** | Market publications and downloadable pesticide-residue results | AgroLink, Farm Sentinel | Current official local source. Build manual/CSV ingestion first; no stable public price JSON API was verified. [12] |
| **Nepal Rastra Bank** | Daily FX, payment-system indicators, financial-inclusion resources and BFI statistics | RemitWise, MeroBazaar dashboards, finance research | NRB's official statistics navigation explicitly includes daily exchange rates and key payment-system indicators. Use an adapter or reviewed import rather than assuming an undocumented API. [19] |
| **Hello Sarkar** | Existing grievance registration/status ecosystem | WardWorks | Primarily a benchmark/ interoperability reference. Do not rebuild its basic function. The portal supports grievance submission and status checking. [28] |
| **MoFAGA** | Official local-government/ municipality navigation and contacts | WardWorks, Relief Inventory | Useful for local-level reference data and partner discovery. [15] |
| **Kathmandu Metropolitan City** | City services, grievance and geoportal links | WardWorks/ urban prototype | Useful as a city-level benchmark demonstrating that a municipal project must differentiate beyond a basic complaint form. [29] |
| **Khalti Payment Gateway** | Nepal-oriented online payment integration | AgroLink, MeroBazaar | Stretch feature only; use official API documentation and merchant/test access. [13] |
| **OpenStreetMap / Nominatim** | Geocoding/map location data | All geospatial projects | Public Nominatim is capped at 1 request/sec, requires attribution/identification, recommends caching and prohibits autocomplete; use a provider abstraction. [11] |

| Source | Data/use | Relevant projects | Access strategy and caveats |
|---|---|---|---|
| **ICIMOD Regional Database System** | Nepal land-cover geospatial datasets | Farm Sentinel, agriculture research | Strong regional research source where government data is insufficient; annual Nepal land-cover datasets are available through ICIMOD's system. [30] |
| **Application-generated datasets** | Road conditions, accessibility audits, facility availability, produce lots, transport observations | Most proposals | Often the most important dataset: capture provenance, timestamps, verification state and consent at creation rather than treating all crowdsourced records as equally reliable. |

A major engineering lesson from these sources is that **data provenance should be visible in the UI**. For example, SafeRoute can label records “BIPAD — updated 10:14” versus “community report — 3 users verified”; AgroLink can label “Kalimati publication” versus “cooperative-entered reference”; ClinicBridge can distinguish “NHFR master data” from “facility-updated 23 minutes ago.” That makes the application more credible and provides a concrete final-year discussion of data quality.

**Resource estimate for the recommended SafeRoute project**

| Resource | Minimum realistic requirement |
|---|---|
| **Team** | One developer; occasional supervisor/domain-user feedback |
| **Effort** | Approximately **180–240 focused hours over 12 weeks** for a polished MVP, documentation and testing |
| **Core skills** | JavaScript/TypeScript, React, Node/Express, MongoDB/Mongoose, REST, async API integration, Git |
| **New skills worth learning** | GeoJSON, `2dsphere` indexes, MapLibre, PWA/offline queues, access control, external-data normalization |
| **Developer tools** | VS Code, Git/GitHub, Postman or Insomnia, MongoDB Compass, browser DevTools |
| **Design/ management** | Figma or paper wireframes; GitHub Issues/Trello-style kanban |
| **Testing** | Vitest, React Testing Library, Supertest, Playwright |
| **Useful libraries** | React Router, TanStack Query, Zod, React Hook Form, Mongoose, MapLibre GL JS, Turf.js, Helmet, rate limiter, structured logger |
| **Hosting for demo** | Vercel frontend + Render Node API + MongoDB Atlas |

| Resource | Minimum realistic requirement |
|---|---|
| **Optional services** | Object storage for report images; email; paid mapping/geocoding only if free/public limits become inadequate |

MongoDB Atlas's free tier currently provides 0.5 GB of storage and limits free clusters to 100 operations per second; that is appropriate for a bounded student demonstration, provided photos are kept in object storage instead of MongoDB documents. [31] Render grants free web services but explicitly warns against production use; idle services spin down after 15 minutes, and the next startup takes about a minute. [24] Vercel currently lists Hobby at $0/month for personal-project use. [32] Thus the **core academic infrastructure can start at approximately $0**, while domain registration, SMS, commercial geocoding/map tiles and additional storage are optional variable costs rather than prerequisites.

**Three-month single-developer plan for SafeRoute Nepal**

| Period | Engineering work | Concrete deliverable |
|---|---|---|
| **Weeks 1–2 — scope and technical proof** | Interview a few prospective users; write problem statement and non-goals; define report categories and risk semantics; test BIPAD ingestion; build low-fidelity screens; initialize React/Express/TypeScript/MongoDB; implement authentication skeleton | Deployed skeleton, BIPAD fixture/import proof, architecture document, database model, clickable map prototype |
| **Weeks 3–4 — authoritative-data map** | Normalize BIPAD events; deduplicate by external ID; GeoJSON/ `2dsphere` indexes; bounding-box/date filters; map layers, legends and event detail cards; source/freshness metadata | Working Nepal map that displays/filter authoritative events and survives upstream outages through cached/seeded data |
| **Weeks 5–6 — community reporting** | Create road-report form; GPS/map pin selection; image upload; expiry; report history; verification voting; moderation queue; confidence calculation; rate limits | End-to-end “observe → report → verify → moderate → display” workflow |
| **Weeks 7–8 — route-risk and offline behavior** | Accept a corridor/polyline; use geospatial intersection/proximity queries; create explainable risk factors; build offline report drafts/queue; add area-alert subscriptions; conduct first user test | Distinctive SafeRoute feature set rather than merely a BIPAD map viewer |
| **Weeks 9–10 — quality hardening** | Authorization tests; input validation; pagination; API error handling; stale-data behavior; accessibility/keyboard testing; responsive UI; admin metrics; automated unit/integration/E2E tests | Release candidate with documented test evidence |

| Period | Engineering work | Concrete deliverable |
|---|---|---|
| **Week 11 — deployment and evaluation** | Vercel/Render/Atlas deployment; environment/ secrets setup; controlled pilot; collect usability observations; fix high-severity bugs; test with API unavailable and slow network | Stable hosted demonstration plus evaluation results |
| **Week 12 — final-year submission polish** | Freeze features; prepare seed/demo script; database export/fallback; architecture/ER screenshots; API documentation; limitations/ future-work chapter; record performance and test results | Viva-ready system, source repository and report |

The **minimum passing scope** by the end of week six should already be defensible: BIPAD synchronization, hazard/incident map, user road report, verification and moderator approval. Route-risk analysis, offline synchronization and alert subscriptions are the differentiating second half. This ordering prevents the common final-year failure mode in which the student spends most of the semester on ambitious extras before achieving one complete user workflow.

The evaluation should also be about more than “the app works.” SafeRoute can measure API synchronization success, median map-query response time on a bounded test dataset, number of steps/ time required to submit a road report, duplicate-report detection behavior, offline-to-online synchronization success, moderation throughput, and task completion in a small usability exercise. Those metrics turn the project from a CRUD demonstration into a defensible software-engineering study.

Finally, the application should retain two explicit limitations in both the UI and thesis: **community reports can be wrong or stale, and route-risk output is advisory rather than a guarantee of safe passage.** The project succeeds academically by integrating authoritative and community information transparently—not by claiming software can certify physical safety. BIPAD remains the authoritative disaster-information integration source, while SafeRoute's original contribution is the localized mobility, verification and decision-support layer built around it. [9]

---


[1] [2] [6] [9] [16] BIPAD API
https://bipadportal.gov.np/api/

[3] कालीमाटी फलफूल तथा तरकारी बजार विकास समिति : सन् १९९५ देखि ...
https://kalimatimarket.gov.np/

[4] [14] [22] NHFR
https://nhfr.mohp.gov.np/

[5] [32] Vercel Pricing: Hobby, Pro, and Enterprise plans
https://vercel.com/pricing

[7] [12] [21] कालीमाटी फलफूल तथा तरकारी बजार विकास समिति : सन् १९९५ देखि नेपाली उपभोक्ता हितमा बजार नियमन गरिराखिएको छ
https://kalimatimarket.gov.np/notice/1303

[8] [23] [28] Hello Sarkar Gunaso - Official Government Complaint Portal Nepal
https://gunaso.opmcm.gov.np/

10 Geospatial Queries - Database Manual - MongoDB Docs
https://www.mongodb.com/docs/manual/geospatial-queries/

11 Nominatim Usage Policy (aka Geocoding Policy)
https://operations.osmfoundation.org/policies/nominatim/

13 Khalti Payment Gateway
https://docs.khalti.com/

15 सङ्घीय मामिला तथा सामान्य प्रशासन मन्त्रालय
https://mofaga.gov.np/

17 20 Integrated Educational Management Information System
https://iemis.cehrd.gov.np/

18 27 Home | Sikai Chautari
https://learning.cehrd.gov.np/en/blog/1008

19 Foreign Exchange Rate - the official site of the Central Bank of Nepal
https://www.nrb.org.np/forex/

24 25 Deploy for Free – Render Docs
https://render.com/docs/free

26 local level | national_population and housing_census_year results
https://censusresults.nsonepal.gov.np/local-level

29 काठमाडौं महानगरपालिका | सांस्कृतिक सहर, समृद्ध महानगर ...
https://kathmandu.gov.np/

30 Home | Regional Database System | ICIMOD
https://rds.icimod.org/

31 Atlas Free Cluster Limits - Atlas - MongoDB Docs
https://www.mongodb.com/docs/atlas/reference/free-shared-limitations/